

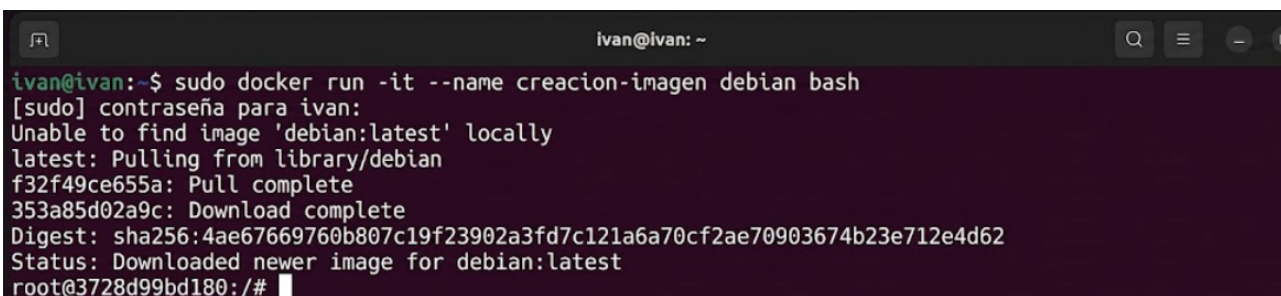
Actividad 6 - Creación de una imagen a partir de un contenedor

Hasta ahora hemos creado contenedores a partir de imágenes de Docker Hub. Para servir nuestra propia aplicación necesitamos crear una imagen personalizada, lo que se conoce como 'dockerizar' una aplicación. La primera forma de hacerlo es partiendo de un contenedor que hayamos modificado.

1. Arrancar un contenedor base

Arrancamos un contenedor interactivo a partir de la imagen de Debian:

```
sudo docker run -it --name creacion-imagen debian bash
```



```
ivan@ivan: ~  
ivan@ivan:~$ sudo docker run -it --name creacion-imagen debian bash  
[sudo] contraseña para ivan:  
Unable to find image 'debian:latest' locally  
latest: Pulling from library/debian  
f32f49ce655a: Pull complete  
353a85d02a9c: Download complete  
Digest: sha256:4ae67669760b807c19f23902a3fd7c121a6a70cf2ae70903674b23e712e4d62  
Status: Downloaded newer image for debian:latest  
root@3728d99bd180:/#
```

2. Modificar el contenedor

Dentro del contenedor, actualizamos e instalamos Apache2:

```
apt update && apt install apache2 -y
```

```
ivan@ivan-pc: ~
root@3728d99bd180:/# apt update && apt install apache2 -y
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [166 kB]
Fetched 10.1 MB in 4s (2400 kB/s)
All packages are up to date.
Installing:
  apache2

Installing dependencies:
  adduser          libcom-err2      libk5crypto3     libperl5.40      linux-sysctl-defaults
  apache2-bin      libcurl4t64      libkeyutils1     libproc2-0       media-types
  apache2-data     libexpat1        libkrb5-3        libpsl5t64       netbase
  apache2-utils    libffi8          libkrb5support0  librtmp1         openssl
  ca-certificates libgdbm-compat4t64 libldap-common   libsasl2-2       perl
  krb5-locales     libgdbm6t64      libldap2         libsasl2-modules perl-modules-5.40
  libapr1t64       libgnutls30t64   liblua5 4-0      libsasl2-modules-db procps
  libaprutil1-dbd-sqlite3 libgpm2          libncursesw6     libssh2-1t64    psmisc
  libaprutil1-ldap libgssapi-krb5-2 libnghttp2-14    libtasn1-6      publicsuffix
  libaprutil1t64   libidn2-0        libnghttp3-9     libunistring5   ssl-cert
  libbrotli1       libjansson4      libp11-kit0      libxml2

Suggested packages:
  liblocale-gettext-perl  ufw          krb5-user          libsasl2-modules-sql
  cron                   www-browser  sensible-utils     perl-doc
  quota                  gdbm-l10n    libsasl2-modules-gssapi-mit  libterm-readline-gnu-perl
  apache2-doc             gnutls-bin  | libsasl2-modules-gssapi-heimdal | libterm-readline-perl-perl
  apache2-suexec-pristine gpm          libsasl2-modules-ldap      make
  | apache2-suexec-custom krb5-doc     libsasl2-modules-otp      libtap-harness-archive-perl

root@3728d99bd180: /#
```

Creamos un index.html personalizado y salimos del contenedor:

```
echo "<h1>Curso Docker</h1>" > /var/www/html/index.html
exit
```

```
ivan@ivan-pc: ~
root@3728d99bd180:/# echo "<h1>Curso Docker</h1>" > /var/www/html/index.html
root@3728d99bd180:/# exit
ivan@ivan-pc:~$
```

3. Crear la nueva imagen con docker commit

Con docker commit generamos una nueva imagen que incluye las capas de la imagen base más los cambios realizados en el contenedor. Si no se indica etiqueta se asigna latest por defecto. Comprobamos con docker images que la nueva imagen aparece en el listado local:

```
sudo docker commit creacion-imagen ivan/myapache2:v1
sudo docker images
```

```
ivan@ivan: ~  
ivan@ivan:~$ sudo docker commit creacion-imagen ivan/myapache2:v1  
sha256:dc7e7e68d17fdf4b6a9c1c1f2913bd9bfb270a3ee871bf9f68e95ff19611f465  
ivan@ivan:~$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
ivan/myapache2:v1	dc7e7e68d17f	347MB	93.6MB	
debian:latest	4ae67669760b	186MB	52.5MB	U
hello-world:latest	0e760fd9bc48	25.9kB	9.49kB	
httpd:2.4	2a7deaaaf357	177MB	47.6MB	
iesgn/guestbook:latest	a1fc3e0652e3	1.32GB	345MB	
iesgn/temperaturas_backend:latest	ad95f9721e81	824MB	221MB	
iesgn/temperaturas_frontend:latest	7a2b7b66f830	863MB	245MB	
mariadb:latest	350261292d01	468MB	111MB	U
mongo:4	52c42cbab240	619MB	176MB	
nginx:latest	800e7c98538c	241MB	66MB	
redis:latest	4d2se2fe601f	204MB	55.3MB	
sdelements/lets-chat:latest	5b923d428176	423MB	108MB	
ubuntu:latest	f3d28607ddd7	160MB	45.3MB	
wordpress:latest	5d2c212561c4	1.08GB	275MB	U

```
ivan@ivan:~$
```

4. Lanzar un contenedor a partir de la nueva imagen

Al crear una imagen con este método no podemos configurar el proceso que se ejecuta por defecto, por lo que hay que indicarlo explícitamente al crear el contenedor. Para arrancar Apache en primer plano usamos `apache2ctl -D FOREGROUND`:

```
sudo docker rm -f creacion-imagen  
sudo docker run -d -p 8080:80 --name servidor_web \  
ivan/myapache2:v1 bash -c 'apache2ctl -D FOREGROUND'  
sudo docker ps -a
```